시니어 헬퍼

매일의 작은 참여를 돕는 디지털 동반자

어르신의 오늘에 ,
친근한 연결을 더하다 .

손자 · 손녀 캐릭터와 함께하는
두뇌 운동 · 대화 · 기억 기록 · 생활 도움

시니어 중심 UX | **음성 + 터치** | **실제 코드 분석**

PROJECT PLANNING × ACCESSIBLE UX × WEB ENGINEERING



기획 의도 | 어려운 기술보다 , 익숙한 경험부터

기능을 많이 제공하는 것보다 스스로 시작하고 끝낼 수 있는 경험을 우선합니다 .

01 정서적 진입

친근하게 시작하기

손자 · 손녀 캐릭터와 인사말로
첫 화면의 거리감을 낮춥니다 .

02 조작의 부담

선택을 단순하게

복잡한 메뉴 대신
네 가지 핵심 행동을 제시합니다 .

03 일상의 연결

기록과 도움으로 이어가기

말과 글로 오늘을 남기고
사진으로 기기 사용을 묻습니다 .

설계 가설 : 친근한 안내와 짧은 활동이 반복 참여를 돕는다 . 실제 효과는 사용자 평가로 확인할 계획입니다 .

타겟 | 디지털 조작에 도움이 필요한 시니어

핵심 사용자는 70 대 이상 어르신으로 설정하되 , 실제 사용 능력의 차이를 고려합니다 .

PRIMARY USER

어르신 본인

작은 글씨 · 복잡한 단계가 부담인 사용자
말하기 · 터치 중 편한 방식을 선택
혼자서도 짧은 활동을 시작하고 마무리

SUPPORTING CONTEXT

가족 · 생활 지원자

초기 화면과 권한 설정을 함께 확인
기록 · 훈련 사용 방법을 안내
향후 현장 평가에 협력할 대상

타겟 정의는 프로젝트 기획 기준입니다 . 보호자 전용 화면 · 원격 모니터링 기능은 현재 범위에 포함하지 않습니다 .

목적과 목표 | 짧은 행동이 일상의 습관이 되도록

제품 목적과 향후 측정할 목표를 구분합니다 . 아래 지표는 현재 성과 수치가 아닙니다 .

사용 시작	첫 핵심 행동까지 단계 축소	첫 행동 완료율 · 소요 시간
반복 참여	짧은 두뇌 활동과 일상 기록	재방문율 · 주간 활동 일수
자기 표현	음성 · 글 중 편한 방식으로 기록	기억 저장 완료율 · 중도 이탈
실패 회복	오류를 알리고 재시도 가능하게	저장 실패율 · 재시도 성공률

핵심 경험 | 네 가지 행동으로 일상과 연결

홈의 2×2 기능 버튼을 중심으로 서비스의 목적을 드러냅니다 .

01

오늘의 두뇌 운동

다섯 종류 · 게임별 10 단계

02

손자 · 손녀에게 물어보기

음성 · 글 질문 · 의도에 따른 안내

03

오늘의 기억

실제 경험 기록 · 음성 다시 듣기

04

기기 사진 찍기

촬영 · 분석 서버 요청 · 쉬운 단계 안내

사용 흐름 | 인사에서 참여 , 기록 , 다시 방문까지

어르신이 행동할 때마다 다음 단계를 쉽게 찾도록 설계합니다 .

STEP 01

인사 · 선택

이름과 캐릭터로
홈에 친숙해지기

STEP 02

짧은 참여

두뇌 운동이나
음성 질문 시작

STEP 03

오늘의 기록

말 · 글로 남기고
실제 목소리 듣기

STEP 04

생활 도움

날씨 확인 · 촬영
필요한 순간 사용

일관된 원칙 : 큰 조작 영역 / 쉬운 한국어 / 오류 안내 / 사용자가 선택하는 권한 요청

화면 설계 | 한 화면에 보이는 주요 행동

모바일 중심 레이아웃과 친숙한 영상 캐릭터를 결합했습니다 .



명확한 시작

이름이 반영된 인사와 손자 · 손녀 선택

큰 행동 버튼

네 가지 기능을 2×2 로 묶어 탐색 부담 완화

상세 정보는 펼쳐서

날씨 상태는 간결하게 , 상세 정보는 누르면 표시

편한 입력 방식

터치 · 글 · 음성을 상황에 따라 선택

첨부 발표 HTML 의 실행 화면 예시 · 날씨 값은 재현용 샘플

두뇌 운동 | 다섯 활동 , 열 단계의 작은 도전

난이도는 숫자 개수 · 경로 폭 · 기억 길이 · 짝 개수로 점진적으로 바뀝니다 .

숫자 순서	누를 숫자 3 개 → 최대 12 개
선 따라 긋기	단계가 오르면 경로 폭 감소 · 굴곡 증가
색상 단추	색상 개수 2 개 → 최대 6 개
소리 · 불빛 기억	기억 순서 길이 3 개 → 12 개
그림 짝 맞추기	짝 개수 2 쌍 → 최대 8 쌍



일상 활동을 돕는 게임 구현입니다 . 의학적 개선 효과는 별도 검증 대상입니다 .

기술 구조 | 화면 · 서비스 · 저장 책임을 나누다

브라우저 기본 기능을 활용하고 , 외부 연결은 서비스 계층에서 처리합니다 .

REACT UI

App · 화면 컴포넌트
홈 / 훈련 / 기억 / 생활 도움

→

SERVICE LAYER

memory · voice · weather
conversation · appliance

→

DATA & INTEGRATION

LocalStorage / Browser APIs
Supabase / Open-Meteo

사진 분석 확장 : React → n8n / 별도 백엔드 → 비전 모델 → 응답 검증 → 쉬운 한국어 안내

핵심 기술 | 기술 선택을 사용자 가치로 연결

기술 이름을 나열하는 대신 , 각 계층의 역할과 선택 이유를 설명합니다 .

React 19 · TypeScript	화면 상태와 데이터 계약	훈련 진행 · 기억 입력 · 응답 형태 관리
Vite 8 · Tailwind CSS 4	빌드 · 스타일 구성	빠른 개발과 모바일 화면 구성
Web Speech · MediaRecorder	인식 · 낭독 · 실제 음성 녹음	말로 입력하고 자신의 목소리 재생
LocalStorage · Supabase	로컬 보관 · 선택적 원격 동기화	서버 미설정 · 일시 실패에도 기록 보존
Open-Meteo · n8n Webhook	날씨 조회 · 사진 분석 연결	생활 맥락 제공과 외부 AI 확장

기록 흐름 | 먼저 보관하고 , 연결되면 동기화

사용자에게 중요한 것은 기록이 사라지지 않고 , 상태를 이해할 수 있는 것입니다 .

01

입력

글 / 녹음
UUID 부여

02

로컬 보관

sync_pending
기기에서 유지

03

인증 · 동기화

서버 발급 ID
음성 업로드 · upsert

04

상태 갱신

성공 : pending 해제
실패 : 보관 · 재시도

삭제도 재시도 대상으로 보관합니다 . 음성 파일을 먼저 삭제한 뒤 DB 기록을 삭제해 , 실패한 파일 경로를 잃지 않도록 합니다 .

핵심 코드 01 | UUID 와 로컬 우선 저장

같은 기록의 ID 를 유지하고 , 동기화 대상 상태를 명시합니다 .

src/services/memoryStorage.ts · L157–166

```
157    saveMemory(item: Omit<MemoryItem, "id" | "created_at">): MemoryItem {
158      const text = item.display_text || formatDisplayText(item.original_text)
159      return this.putMemory({
160        ...item,
161        id: crypto.randomUUID(),
162        display_text: text,
163        summary: text,
164        created_at: new Date().toISOString(),
165        sync_pending: true,
166      })
```

실제 코드 발췌 (줄바꿈 조정) • 01 / 09 원본 코드 열기 ↗

무엇을 구현했나

기록 생성 시 UUID· 생성 시각 · 동기화 대기 상태를 함께 저장

사용자에게 연결되는 결과

기억을 먼저 기기에 보관하고 원격 연결을 준비

설계 판단

서버 연결 여부와 입력 경험을 분리합니다 . 로컬 저장 실패는 별도로 처리합니다 .

핵심 코드 02 | 동기화 작업의 순서 보장

같은 탭에서 저장 · 조회 · 삭제가 교차하며 캐시를 덮어쓰지 않도록 합니다 .

src/services/memoryService.ts · L18–21, L77–81

```
18  function run<T>(action: () => Promise<T>): Promise<T> {
19    const result = operations.then(action)
20    operations = result.catch(() => {})
21    return result
    // …
77      const { error } = await supabase
78        .from("memories")
79        .upsert(row, { onConflict: "id" })
80      if (error) throw error
81      memoryStorage.putMemory({ ...item, sync_pending: false })
```

실제 코드 발췌 (줄바꿈 조정) • 02 / 09 원본 코드 열기 ↗

무엇을 구현했나

Promise 체인으로 작업을 직렬화하고 ID 기준 upsert 수행

사용자에게 연결되는 결과

재시도해도 같은 기억 ID 를 사용하며 성공 후 대기를 해제

설계 판단

클라이언트 작업 순서와 중복 업로드 대응을 분리해 관리합니다 .

핵심 코드 03 | 말한 문장을 이어서 기록

음성 인식 텍스트와 실제 녹음 파일은 서로 다른 데이터로 관리합니다 .

src/services/voiceRecordService.ts · L47–61

```
47          this.recognition = new Recognition()
48          this.recognition.lang = "ko-KR"
49          this.recognition.continuous = true
50          this.recognition.interimResults = true
51          this.recognition.onresult = (event: any) => {
52            if (generation !== this.generation) return
53            // continuous  결과 목록 전체를 사용해 이전 확정 문장을 유지합니다 .
54            this.transcript = Array.from(
55              event.results as ArrayLike<any>,
56              (result) => result[0].transcript,
57            )
58              .join(" ")
59              .trim()
60            onTranscriptUpdate?.(this.transcript)
61          }
```

실제 코드 발췌 (줄바꿈 조정) • 03 / 09 원본 코드 열기 ↗

무엇을 구현했나

한국어 연속 인식 결과 목록을 다시 모아 전체 문장 유지

사용자에게 연결되는 결과

앞서 말한 문장이 사라지지 않고 입력란에 누적

설계 판단

MediaRecorder 로 실제 음성도 저장합니다 . 브라우저 미지원 · 인식 실패 시 직접 입력으로 보완합니다 .

핵심 코드 04 | 질문 의도를 기능으로 연결

키워드 기반 분기로 날씨와 가전 질문을 구분합니다 .

src/services/aiGrandchild.ts · L23–28

```
 23    const weatherQuestion = ["날씨", "기온", "습도", "체감온도", "비가", "비와", "추
워",
     "더워"].some(word => lower.includes(word)) ||
 24      (lower.includes("온도") && !["에어컨", "보일러", "리모컨", "냉장고", "전자레인지",
"밥솥",
     "기기"].some(word => lower.includes(word)));
 25    if (weatherQuestion) return { answer: weatherAnswer(), isApplianceQuestion: false
     };
 26
 27    // 1. 전자기기 / 가전제품 관련 질문 감지
 28    const isAppliance = APPLIANCE_KEYWORDS.some((kw) => lower.includes(kw));
```

실제 코드 발췌 (줄바꿈 조정) • 04 / 09 원본 코드 열기 ↗

무엇을 구현했나

날씨 의도를 우선 확인하고 기기 온도 질문과 구분

사용자에게 연결되는 결과

날씨는 최신 캐시로 답하고 , 가전 질문은 촬영을 제안

설계 판단

현재 일반 대화는 준비된 응답입니다 . 생성형 대화 모델 연동은 향후 확장 과제입니다 .

핵심 코드 05 | 사진 분석 결과를 검증

확인되지 않은 응답을 기기 조작 안내로 표시하지 않도록 합니다 .

src/services/applianceService.ts · L84–92

```
84      if (data?.success !== true || data.needs_new_photo === true) {
85        return { success: false, device_name: "", instructions: [], needs_new_photo:
    data?.needs_new_photo === true,
86          error_guide: typeof data?.error_guide === "string" ? data.error_guide :
    " 사진에서 기기를 확인하지 못했어요 .  다시 찍어주세요 ." };
87      }
88      if (typeof data.device_name !== "string" || !data.device_name.trim() ||
89        !Array.isArray(data.instructions) || data.instructions.length < 1 ||
    data.instructions.length > 3 ||
90        !data.instructions.every((step: unknown) => typeof step === "string" &&
    step.trim().length > 0)) {
91        throw new Error(" 분석 응답 형식이 올바르지 않습니다 .");
92      }
```

실제 코드 발췌 (줄바꿈 조정) • 05 / 09 원본 코드 열기 ↗

무엇을 구현했나

성공 여부 · 재촬영 여부 · 기기명 ·1~3 단계 안내 형식 확인

사용자에게 연결되는 결과

실패 시 오류 또는 재촬영을 안내하고 임의의 사용법을 만들지 않음

설계 판단

프런트엔드는 응답 계약을 검증합니다 . 실제 기기 식별 정확성은 운영 모델로 별도 평가해야 합니다 .

핵심 코드 06 | 늦게 온 날씨 응답 차단

지역을 빠르게 바꿔도 이전 요청 결과가 새 지역을 덮지 않도록 합니다 .

src/hooks/useWeather.ts · L28–47

```
28      request.current?.abort()
29      const controller = new AbortController()
30      const id = ++sequence.current
31      request.current = controller
32      setLoading(true)
33      setError("")
34      const timeout = setTimeout(() => controller.abort(), 8000)
35      try {
36        const result = await fetchCurrentWeather(location, controller.signal)
37        if (id !== sequence.current) return
38        setWeather(result)
39        cacheWeather(result)
40        setNow(Date.now())
41      } catch {
42        if (id === sequence.current)
43          setError(" 날씨를 가져오지 못했어요 .  잠시 뒤 새로고침해 주세요 .")
44      } finally {
45        clearTimeout(timeout)
46        if (id === sequence.current) setLoading(false)
```

실제 코드 발췌 (줄바꿈 조정) • 06 / 09 원본 코드 열기 ↗

무엇을 구현했나

이전 요청 취소 + 요청 순번 확인 + 8 초 타임아웃

사용자에게 연결되는 결과

선택한 지역의 날씨와 화면 상태를 일치시킴

설계 판단

요청 실패는 마지막 자료를 유지하며 표시합니다 . 30 분 초과 자료는 오래된 정보로 구분합니다 .

핵심 코드 07 | 서버가 발급한 사용자 ID

원격 데이터 소유자는 브라우저가 임의 생성한 ID 와 구분합니다 .

src/services/userService.ts · L27–37

```
 27        const { data: session, error: sessionError } =
 28          await supabase.auth.getSession()
 29        if (sessionError) throw sessionError
 30        if (session.session) {
 31          const { data, error } = await supabase.auth.getUser()
 32          if (error) throw error
 33          return data.user.id
 34        }
 35        const { data, error } = await
supabase.auth.signInAnonymously()
 36        if (error) throw error
 37        return data.user?.id ?? null
```

실제 코드 발췌 (줄바꿈 조정) • 07 / 09 원본 코드 열기 ↗

무엇을 구현했나

기존 세션의 사용자를 확인하고 없으면 익명 인증

사용자에게 연결되는 결과

가입 절차 부담을 낮추면서 인증된 사용자로 원격 저장

설계 판단

익명 세션은 브라우저에 연결됩니다 . 기기 간 복구 · 공유는 영구 계정 설계가 필요합니다 .

핵심 코드 08 | 소유자 정책과 비공개 음성

DB 접근 정책과 파일 경로 정책을 함께 적용합니다 .

supabase_schema.sql · L110–111, L141–143

```
110  CREATE POLICY "owner_access" ON public.memories FOR ALL TO authenticated
111  USING (user_id = (select auth.uid())::text) WITH CHECK (user_id = (select
     auth.uid())::text);
     // …
141  CREATE POLICY "memory_audio_owner" ON storage.objects FOR ALL TO authenticated
142  USING (bucket_id = 'memory-audio' AND (storage.foldername(name))[1] = (select
     auth.uid())::text)
143  WITH CHECK (bucket_id = 'memory-audio' AND (storage.foldername(name))[1] = (select
     auth.uid())::text);
```

실제 코드 발췌 (줄바꿈 조정) • 08 / 09 원본 코드 열기 ↗

무엇을 구현했나

auth.uid() 로 DB 소유자와 음성 폴더 소유자 제한

사용자에게 연결되는 결과

다른 사용자의 기억과 음성에 대한 접근 차단

설계 판단

음성은 비공개 버킷에 보관하며 재생용 URL 은 300 초로 발급합니다 . 실제 Supabase 설정은 별도 적용합니다 .

핵심 코드 09 | 끝점 터치만으로 통과하지 않기

선 긋기 활동은 시작 위치와 경로 진행을 검사합니다 .

src/App.tsx · L703–712, L738–741

```
703    const handlePointerDown = (e: React.PointerEvent<HTMLCanvasElement>) => {
704      const canvas = e.currentTarget;
705      const rect = canvas.getBoundingClientRect();
706      const x = e.clientX - rect.left;
707      const y = e.clientY - rect.top;
708      if (Math.hypot(x - 45, y - canvas.height / 2) > 24) return;
709      canvas.setPointerCapture(e.pointerId);
710      isDrawingRef.current = true;
711      previousPoint.current = { x: 45, y: canvas.height / 2 };
712      progress.current = 45;
     // …
738      if (progress.current >= endX - 8 && Math.hypot(x - endX, y - endY) < 24)
{
739        isDrawingRef.current = false;
740        soundSuccess(); buzz(45); onLevelComplete();
741      }
```

실제 코드 발췌 (줄바꿈 조정) • 09 / 09 원본 코드 열기 ↗

무엇을 구현했나

시작점 거리 확인 · 포인터 캡처 · 진행률과 도착점 확인

사용자에게 연결되는 결과

시작부터 경로를 따라 이동한 경우에만 단계 완료

설계 판단

이동 중에는 보간한 점의 경로 이탈 · 과도한 역행도 검사합니다 . 핵심은 활동 의도에 맞는 완료 조건입니다 .

검증 결과 | 실행 가능한 구현 근거를 남기다

동일한 코드 기준에서 빌드와 기존 회귀 검사를 실행했습니다 .

20

브라우저 테스트 통과

로컬 기억 · 음성 · 날씨 · 훈련
동기화 재시도 · 영상 동작

3

보안 · 배포 테스트 통과

PGlite 접근 정책 ·SQL 재실행
환경변수 · 실제 미디어 검사

PASS

타입 검사 + 배포 빌드

npm run build:deploy
프로덕션 정적 파일 생성

외부 응답과 장치 권한은 일부 모킹합니다 . 실제 Supabase· 사진 분석 정확도 · 고령 사용자 사용성 검증은 별도 과제입니다 .

현재 범위 | 구현과 운영 연결을 함께 설명

발표에서 보여줄 수 있는 동작과 추가 준비가 필요한 부분을 분명히 합니다 .

기본 앱 · 훈련 · 기억	로컬 실행 · 브라우저 검사 확인	구현 · 검증
날씨	조회 · 캐시 · 지역 선택 구현 / 실시간 접근은 환경별 확인	연결 확인
Supabase 동기화	인증 ·RLS· 음성 저장 코드 / 모킹 ·PGlite 검사	운영 설정
사진 분석	촬영 · 웹훅 · 응답 검증 / 실제 분석 백엔드 별도	서버 연결
대화 · 사용자 효과	준비된 응답 / 실제 사용자 성과는 미측정	향후 검증

다음 목표 | 운영 연결 , 사용자 검증 , 개인화

기술적 동작에서 실제 사용자의 지속적인 참여로 검증 범위를 넓힙니다 .

01 운영 안정화

실환경 연결

Supabase 익명 인증 ·RLS 적용
사진 분석 HTTPS·CORS 확인
실기기 권한 · 오프라인 검증

02 사용성 평가

어르신과 함께 확인

핵심 행동 완료율 · 소요 시간
오류 경험 · 재시도 성공률
글씨 · 버튼 · 문구 개선

03 확장 검토

기록의 연속성

영구 로그인 · 계정 복구
기기 간 기억 연동
대화 모델 · 개인화 검토

작은 화면 속 경험을 , 짧고 선명하게

프로젝트 소개 숏폼으로 시니어 헬퍼를 만나보세요 .

설명에서 경험으로
이어지는 한 번의 클릭

발표 → 숏폼 → 실제 앱
관심을 서비스 체험으로 연결합니다 .

YouTube Shorts 보러 가기 ↗

youtube.com/shorts/F8F46lj6yW8

휴대폰으로 QR 스캔

직접 사용해 보세요

발표 마지막은 네 가지 핵심 경험을 따라가는 짧은 시연으로 마무리합니다 .

01 손자 · 손녀 선택 → 오늘의 두뇌 운동

02 오늘의 기억 → 글로 저장하고 다시 열기

03 날씨 상세 → 지역 선택과 상태 확인

04 기기 사진 → 촬영과 연결 상태 안내

서비스 열기 ↗

GitHub 코드 보기 ↗



senior-helper-seven.vercel.app

자료와 분석 근거

참고 자료의 구성은 활용하고 , 프로젝트 사실은 현재 저장소 코드를 기준으로 작성했습니다 .

참고 01 · where_u_go.pptx	기획 → 아키텍처 → 코드 → 검증 → 로드맵 구성과 코드 해설 방식 참고
참고 02 · 첨부 HTML 발표 자료	시니어 헬퍼의 기존 소개 내용과 모바일 홈 예시 화면 참고
주요 소스 · 현재 저장소	src/App.tsx · services · hooks · lib · supabase_schema.sql · tests
검증 · 프로젝트 명령	npm test / npm run build:deploy · 코드 기준 4f16596
연결 · 사용자 제공 숏폼	YouTube Shorts F8F46lj6yV8 · 클릭 버튼과 QR 코드 제공

코드 슬라이드의 ‘원본 코드 열기’에서 해당 커밋의 실제 코드를 확인할 수 있습니다 .